

LECTURE NOTES

Information Systems Analysis and Design

Object-Oriented Design

Dr. Dina Fitria Murad, S.Kom., M.Kom.



Learning Outcomes

LO 4: Mengevaluasi pengembangan sistem, manajemen proyek, dan desain berorientasi objek, dan menerapkan sistem baru

Outline Materi

1. Bridging from Analysis to Implementation
2. Fundamental Principles for Good Design
3. Use Case Realization
4. Multilayer Design

9.1

Bridging from Analysis to Implementation

Apa itu desain berorientasi objek? Ini adalah proses di mana satu set detail model desain berorientasi objek dibangun, yang kemudian digunakan oleh pembuat program untuk menulis dan menguji program untuk sistem baru. Desain sistem adalah jembatan antara kebutuhan pengguna dan pemrograman sistem baru. Satu kekuatan dari pendekatan berorientasi objek adalah bahwa model desain seringkali hanya ekstensi dari model requirement. Jelas, jauh lebih mudah untuk memperluas model yang sudah ada daripada membuat model desain yang sama sekali baru. Seringkali, pengembang suka melewati langkah model desain; namun, ini adalah praktik yang baik untuk membuat model desain dan tidak hanya terjun ke coding. Sama seperti seorang pembangun tidak membangun sesuatu yang lebih besar daripada rumah anjing atau gudang tanpa set cetak biru, pengembang sistem yang baik tidak akan pernah mencoba mengembangkan sistem besar tanpa seperangkat model desain.

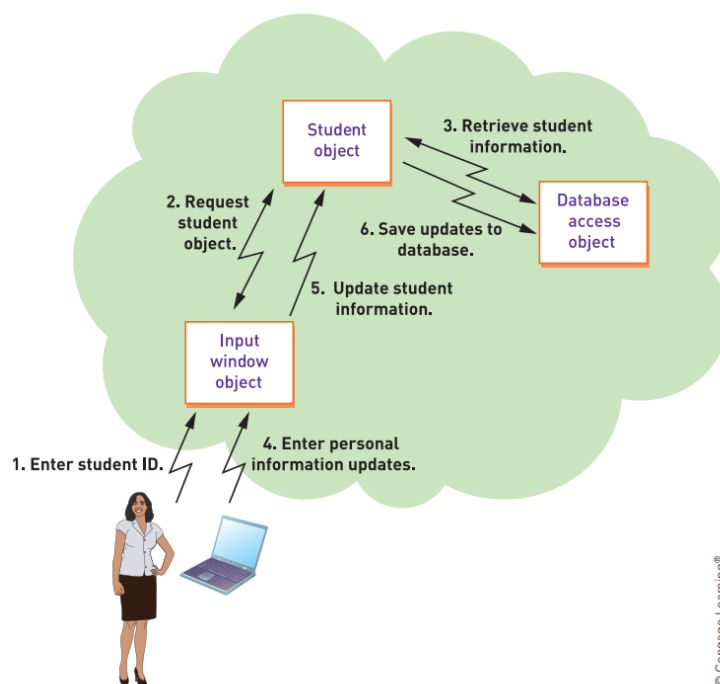
Metode Agile adalah pendekatan pengembangan perangkat lunak yang fleksibel dan adaptif. Metode ini menekankan kolaborasi tim, komunikasi yang efektif, dan pengiriman perangkat lunak dalam iterasi pendek yang disebut sprint. Metode Agile memungkinkan perubahan kebutuhan pengguna selama proses pengembangan dan memungkinkan tim untuk merespons perubahan dengan cepat. Salah satu prinsip Agile terkait pendekatan pengembangan adaptif ini menekankan pada pembuatan model hanya jika mereka memiliki makna dan diperlukan. Terkadang, pengembang baru salah mengartikan pedoman ini yang berarti mereka tidak perlu mengembangkan model desain sama sekali. Model desain mungkin tidak diformalkan menjadi satu set dokumen dan diagram yang komprehensif, tetapi pasti diperlukan.

Mengembangkan layer tanpa melakukan desain sebanding dengan menulis makalah layer tanpa garis besar. Anda bisa duduk dan mulai menulis. Namun, jika Anda menginginkan makalah yang kohesif, lengkap, dan komprehensif, Anda harus menulis garis besar terlebih dahulu. Anda dapat menulis makalah yang rumit tanpa garis besar, tetapi kemungkinan besar, itu akan terputus-putus, sulit diikuti, dan kehilangan poin penting—dan akan mendapat nilai rendah! Garis besar dapat ditulis di atas kertas, tetapi proses memikirkannya dan menuliskannya memungkinkan penulis untuk memastikan bahwa itu kohesif.

Program berorientasi objek terdiri dari layer dari objek komputasi. Setiap objek memiliki data dan logika program yang dikapsulasi di dalam dirinya sendiri. Analisis mendefinisikan struktur logika program dan bidang data dengan mendefinisikan Class.

Definisi class menggambarkan struktur atau seperti template dari objek yang dieksekusi. Objek itu sendiri tidak muncul sampai program mulai dieksekusi. Ini disebut instantiasi class—yaitu, membuat instance (objek) berdasarkan template yang disediakan oleh definisi class. Sebuah program berorientasi objek terdiri dari satu set objek instantiated yang bekerja sama untuk mencapai suatu hasil. Objek-objek ini bekerja sama dengan mengirimkan pesan satu sama lain dan bekerja sama untuk mendukung fungsi program utama.

Gambar 9.1 menggambarkan bagaimana program berorientasi objek bekerja. Program termasuk objek jendela yang menampilkan layer untuk memasukkan ID siswa dan informasi lainnya (pesan 1). Dari perspektif pengguna, objek ini muncul sebagai bentuk di layer. Setelah ID siswa dimasukkan, objek Window mengirimkan pesan (pesan 2) ke class Siswa untuk menyuruhnya membuat objek Siswa baru (contoh) dalam program. Objek Siswa baru akan tahu bahwa ia membutuhkan lebih banyak data untuk beberapa atributnya berdasarkan logika internal di class siswa, sehingga akan mengirim pesan ke objek Akses Basis Data yang meminta data dari basis data (pesan 3). Setelah itu benar-benar dipakai dengan semua data yang diperlukan, objek Siswa mengirimkan informasi layer ke objek Window untuk ditampilkan itu di layer. Pengguna siswa kemudian memasukkan pembaruan ke informasi pribadinya (pesan 4), dan urutan pesan lainnya dikirim untuk memperbarui objek Siswa dalam program, yang meneruskan informasi ke Akses objek Basis Data dan menuliskannya ke database.



Gambar 9.1. Object-oriented event-driven program flow

Pengembangan perangkat lunak dalam sistem informasi memiliki peran kunci dalam meningkatkan efisiensi organisasi dan memenuhi kebutuhan pengguna. Metode pengembangan yang efektif dan pemahaman yang baik tentang kebutuhan pengguna sangat penting dalam mencapai tujuan tersebut.

Terdapat beberapa model pengembangan perangkat lunak yang digunakan dalam proses pengembangan. Beberapa model yang populer antara lain:

- Model Waterfall: Model ini adalah model populer dalam pengembangan perangkat lunak yang sistematis atau sekuensial. Proses pengembangan dilakukan secara berurutan atau linear, dimana setiap tahapan harus selesai sebelum melanjutkan tahapan berikutnya. Tahapan dalam model Waterfall meliputi analisis persyaratan, desain, pengkodean, pengujian, dan pemeliharaan .

- Model RAD (Rapid Application Development): Model pengembangan RAD juga dikenal sebagai model pendekatan berbasis objek. Metode RAD menggunakan pendekatan iteratif dalam mengembangkan sistem, dimana sistem dikonstruksikan di awal tahap pengembangan untuk menetapkan kebutuhan pengguna. Metode RAD memungkinkan pengembangan aplikasi dengan waktu yang relatif lebih cepat, sekitar 30-90 hari, dan sangat mementingkan keterlibatan pengguna dalam proses analisis dan perancangan .
- Metode Agile: Metode Agile adalah pendekatan pengembangan perangkat lunak yang fleksibel dan adaptif. Metode ini menekankan kolaborasi tim, komunikasi yang efektif, dan pengiriman perangkat lunak dalam iterasi pendek yang disebut sprint. Metode Agile memungkinkan perubahan kebutuhan pengguna selama proses pengembangan dan memungkinkan tim untuk merespons perubahan dengan cepat.
- Metode Spiral: Metode Spiral digunakan dalam pengembangan perangkat lunak berskala besar yang membutuhkan sistem yang kompleks. Metode ini melibatkan analisis mendalam mengenai tingkat risiko dan keberhasilan pengembangan. Proses pengembangan dilakukan dalam lima langkah, yaitu komunikasi, perencanaan, analisis dan perancangan, konstruksi, dan evaluasi
- Metode DevOps: Metode DevOps merupakan pendekatan yang mengkombinasikan proses pengembangan (development) dan operasi (operations). Metode ini bertujuan untuk mengintegrasikan pengembangan perangkat lunak dengan infrastruktur secara kolaboratif. DevOps memungkinkan pengembang untuk merancang dan mengirimkan perangkat lunak dengan lebih efisien.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

- Apa yang dimaksud dengan pendekatan berorientasi objek? Apa perbedaannya dengan pendekatan terstruktur?
- Apa yang dimaksud dengan pendekatan pengembangan adaptif? Contohnya seperti apa?
- Berdasarkan beberapa model yang populer, model mana yang menurut kalian paling banyak diminati di industry saat ini? Kenapa?

9.2

Fundamental Principles for Good Design

Dengan sesuatu yang serumit aplikasi perangkat lunak, mungkin ada jumlah tak terbatas desain dan solusi pemrograman yang akan bekerja. Beberapa solusi akan menjadi solusi "baik", sementara banyak solusi lain akan dipertimbangkan menjadi solusi "buruk".

Karakteristik ini menentukan kualitas desain. Dengan kata lain, mereka menunjukkan "kebaikan" dari desain terperinci dan implementasi dari sistem baru. Meskipun memahami dan menggunakan prinsip-prinsip desain tersebut tidak akan menjamin desain menjadi berkualitas tinggi, atau pun dengan menerapkan prinsip-prinsip ini akan meningkatkan probabilitas bahwa desainnya solid. Mengabaikan prinsip-prinsip berikut hampir pasti memastikan desain perangkat lunak yang kurang optimal.

Pembahasan prinsip-prinsip desain yang baik adalah titik awal untuk memahami bagaimana merancang perangkat lunak berkualitas tinggi. Namun, itu bukan diskusi yang komprehensif. Ada banyak buku dan artikel bagus tentang prinsip-prinsip desain berorientasi objek yang baik, yang lebih jauh menjelaskan prinsip dan teknik desain yang baik. Terdapat banyak perdebatan tentang seberapa banyak dasar-dasar desain yang ada. Beberapa desainer mengatakan ada tujuh, yang lain dua belas, dan yang lainnya di antara keduanya. Tetapi ketika berbicara tentang dasar-dasar desain, angka bukanlah sesuatu yang mutlak.

Dasar-dasar desain memberi desainer beberapa pedoman tentang bagaimana merancang komposisi yang menarik secara visual yang menciptakan pengalaman pengguna yang luar biasa. Dengan mengikuti prinsip dasar desain seperti hierarki, keseimbangan, kesatuan, dan variasi untuk dapat membuat produk digital dan desain grafis yang disukai orang. Dasar desain yang harus diketahui oleh setiap desainer grafis:

- Hierarchy

Hierarchy visual adalah bagian penting dari desain yang baik. Jika semua objek yang ada di lembar kerja memiliki kepentingan yang sama, maka tidak ada yang menonjol. Perlu menggunakan bahasa visual untuk memberi tahu orang apa yang menjadi pusat perhatian pertama, kedua, ketiga, dll. Dalam membuat hierarki visual dapat melalui skala (ukuran relatif dari suatu elemen) dan warna. Hierarchy tipografi dapat dibuat dengan menggunakan tipografi, ukuran, dan bobot font yang berbeda. Pada intinya adalah untuk memastikan elemen terpenting menonjol pada halaman dibandingkan dengan elemen yang lain

- Balance

Keseimbangan dalam komposisi dapat tercapai dengan beberapa cara berbeda. Keseimbangan simetris adalah yang paling mudah. Hal ini tercapai ketika elemen di kedua sisi sumbu vertikal pada dasarnya sama. Misalnya, dua blok teks di kedua sisi halaman akan menciptakan keseimbangan simetris, meskipun konten blok tersebut tidak identik.

Keseimbangan asimetris tercapai ketika elemen di kedua sisi sumbu pusat tidak sama. Misalnya, ketika memiliki gambar besar di satu sisi yang diimbangi dengan teks yang menonjol di sisi lain. Hal ini juga dapat tercapai ketika sumbu vertikal yang membagi dua elemen tidak ditempatkan

langsung di tengah halaman. Dalam hal ini, elemen yang lebih sempit harus memiliki bobot visual yang “lebih berat” daripada elemen yang lebih lebar untuk mendapatkan tampilan yang seimbang. Apapun jenis teknik keseimbangan yang digunakan, hasilnya akan terasa sesuai. Hal ini seharusnya membuat yang melihatnya terasa selaras dan justru tidak membuat merasa tidak nyaman.

- **Alignment**

Alignment mengacu pada bagaimana suatu teks atau elemen grafis sejajar pada halaman. Mengacu pada keselarasan dalam kaitannya dengan seluruh komposisi (rata kiri, tengah, atau kanan) serta penyelarasan satu dengan yang lain. Ketika elemen tidak selaras, terutama dalam kaitannya satu sama lain akan menambah kesan kekacauan pada komposisi. Akan terlihat acak-acakan dan berantakan. Ketika sampai pada prinsip dasar desain ini, pastikan untuk menyelaraskan elemen dengan benar dalam hubungannya satu dengan yang lain, dan konsisten dengan penyelarasan dari berbagai elemen, selalu menjadikan headlines sebagai pusatnya.

- **Emphasis**

Penekanan adalah bagian dari desain yang menarik perhatian pengguna—dengan kata lain, poin utama. Idealnya, hal ini harus menjadi bagian terpenting dari desain, apakah itu judul, gambar, atau CTA (Call to Action). Tapi hal itu terkadang tidak menjadi kenyataan. Seorang desainer yang tidak berpengalaman mungkin secara tidak sengaja memberi penekanan pada bagian halaman yang salah, yang justru akan membuat kebingungan di pihak pengguna. Sebaiknya memastikan pada bagian yang ingin ditonjolkan atau ditekankan pada kesan pertama pengguna. Dengan melakukannya melalui proporsi elemen, white space, warna, bayangan, pola, atau teknik lainnya.

- **Proportion**

Proporsi, disebut juga sebagai skala, adalah ukuran relatif objek dalam sebuah desain. Elemen yang lebih besar dalam kaitannya dengan yang lain akan lebih menonjol dan tampak lebih penting bagi pengguna. Elemen yang lebih kecil tampak kurang penting. Sebaiknya menggunakan proporsi untuk menciptakan minat visual dengan menarik perhatian pemirsa ke elemen visual tertentu dalam desain. Selalu diingat bahwa terlalu banyak perbedaan skala dari elemen dapat membuat elemen terkecil hilang, atau membuat elemen terbesar tampak besar dan terkesan main main

- **Movement**

Gerakan mengacu pada cara mata pengguna bergerak dalam melihat komposisi desain. Desain yang dinamis mendorong banyak gerakan mata, sementara desain statis lebih sedikit. Seorang desainer yang baik dapat melakukan batasan, membuat kontrol dari elemen mana saja yang menjadi fokus pengguna dengan menempatkannya di sepanjang jalur dari pola gerakan mata. Penting untuk membiasakan diri dengan pola gerakan mata yang paling umum, pola F dan Z, dan pola kue lapis. Pola F- dan Z lebih umum pada halaman dengan banyak gambar, sedangkan pola kue lapis difasilitasi oleh banyak teks dengan judul dan subjudul.

- **Negative Space**

Ruang negatif dalam sebuah desain, disebut juga sebagai ruang kosong, adalah ruang yang tidak memiliki elemen desain (selain karena warna latar belakang atau pola atau tekstur halus). Sebagai prinsip desain, ruang negatif sangat penting karena memberikan elemen-elemen dalam ruang

komposisi untuk bernafas. Jika tanpa spasi, halaman terlihat berantakan dan sulit dinavigasi. Selalu pastikan untuk memberikan ruang di sekitar elemen di halaman, terutama yang paling penting. Ruang kosong ini akan membuat lebih menonjol dan memfasilitasi pengalaman pengguna yang lebih baik.

■ Contrast

Kontras dapat dicapai melalui penggunaan warna, bentuk, ukuran, atau sifat elemen yang serupa, dan mengacu pada perbedaan di antara mereka. Warna yang kontras sering kali menjadi hal pertama yang dipikirkan orang, tetapi perbedaan ukuran elemen, bentuk, atau properti lainnya juga dapat menciptakan kontras. Kontras memiliki dua kegunaan yang sangat penting. Pertama, memungkinkan dalam membuat elemen menjadi menonjol satu dengan yang lain. Kurangnya kontras akan menghasilkan desain yang hanya memiliki satu warna latar belakang tanpa elemen lain yang terlihat — bukan desain fungsional. Sebuah desain di mana dapat terlihat elemen yang berbeda secara otomatis memiliki beberapa tingkat kontras.

Kegunaan penting dari kontras yang kedua adalah aksesibilitas. Kontras yang memadai antara elemen, terutama teks dan latar belakangnya, sangat penting untuk menciptakan desain yang dapat diakses. Orang dengan gangguan penglihatan dapat mengalami kesulitan membaca teks pada layar yang terlalu kecil atau tidak memiliki kontras warna yang memadai.

■ Repetition

Pengulangan mengacu pada penggunaan elemen yang sama atau serupa di seluruh desain, baik dalam pola teratur atau tidak beraturan. Hal ini digunakan untuk memperkuat elemen tertentu sambil juga memberikan rasa kesatuan dan kontinuitas pada desain. Pengulangan dapat digunakan untuk menciptakan ritme, yang membantu menggerakkan pengguna melalui desain. Gunakan pengulangan dengan cara yang sederhana—seperti menggunakan ikon yang sama, dalam sebuah pola latar belakang, atau melalui hal-hal seperti mengatur semua foto dengan cara yang sama. Bereksperimenlah dengan pengulangan, khususnya pengulangan tidak beraturan, untuk menciptakan suatu gerakan, tetapi berhati-hatilah karena terlalu banyak pengulangan secara reguler justru memiliki efek sebaliknya dan membuat desain terasa statis.

■ Variety

Variasi adalah suatu bumbu kehidupan, bukankah begitu? Hal yang sama juga berlaku dalam desain. Jika pengulangan menambah rasa harmoni pada desain, variasi membuatnya tetap menarik dan mencegah pengguna merasa bosan. Ciptakan variasi dengan menambahkan elemen unik atau tak terduga ke desain. Variasi dapat digunakan untuk menarik perhatian pengguna ke elemen atau area tertentu dari desain, dan membuatnya menonjol.

■ Unity

Unity memberikan desain dan rasa akan keharmonisan, baik secara visual maupun konseptual. Unity penting karena membuat pengguna merasa nyaman saat menavigasi desain. Semuanya tampak pada tempatnya dan tidak ada elemen yang menonjol secara negatif. Unity dapat diciptakan dengan berbagai cara. Metode utama adalah melalui:

1. Proximity — Elemen yang dekat satu sama lain tampak seperti grup terpadu
2. Alignment — Elemen yang disejajarkan satu sama lain tampaknya terkait

3. Repetition — Elemen yang diulang juga menciptakan rasa kesatuan, terutama jika pengulangan itu dilakukan dalam pola yang teratur.

Menerapkan prinsip-prinsip desain dapat dilakukan dengan mempelajari dan mengikuti prinsip desain yang mapan dalam desain grafis memungkinkan dalam membuat desain yang lebih kohesif yang menyenangkan pengguna dan menawarkan pengalaman pengguna yang luar biasa. Mengabaikan prinsip-prinsip desain ini harus dilakukan secara hati-hati, dan hanya setelah memiliki pemahaman yang menyeluruh tentang prinsip-prinsip tersebut dan tujuan yang disajikan.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

- Bagaimana seorang disainer merancang komposisi yang menarik secara visual yang menciptakan pengalaman pengguna yang luar biasa?
- Apakah semua prinsip desain harus diterapkan pada saat mendisain sebuah apps? Kenapa?

Belajar Mandiri



Pelajari secara mandiri terkait topik Object-Oriented Design pada beberapa sumber referensi berikut ini untuk mempermudah kalian memahami nya:

- Object-Oriented Design, <https://www.javatpoint.com/software-engineering-object-oriented-design>
- Object-Oriented Design, https://www.youtube.com/watch?v=fJW65Wo7IHI&list=PLGLfVvz_LVvS5P7khyR4xDp7T9lCk9Pg

9.3 Usecase Realization

Use Case adalah simbol yang menggambarkan suatu Kegiatan (Aktifitas) yang terjadi pada sistem. Use Case Realization adalah simbol yang menggambarkan suatu Kegiatan (Aktifitas) yang terjadi pada sistem dan hanya dapat dilihat di dalam sistem. Use case realization memberikan sudut pandang dari dalam (internal view) terhadap use-case tersebut, yakni menentukan bagaimana pekerjaan (dalam business usecase) tersebut diatur dan dimainkan untuk mencapai hasil yang diinginkan.

Realisasi use case menyediakan konstruksi untuk mengatur artefak yang menunjukkan bagaimana desain fisik suatu sistem mendukung perilaku bisnis logis yang digariskan oleh use case. Untuk menunjukkan ketertelusuran antara desain logis dan fisik, dalam diagram use case setiap use case digambarkan sebagai oval yang ditunjukkan dengan batas garis padat. Kemudian untuk setiap use case, Anda dapat menampilkan realisasi use case dalam bentuk oval dengan batas garis putus-putus. Use case dan realisasi use case terkait dihubungkan menggunakan ketergantungan realisasi yang ditampilkan dalam UML sebagai garis putus-putus dengan panah segitiga di ujung sesuai dengan elemen yang direalisasikan (use case).

Setiap realisasi use case akan menentukan desain fisik dalam hal kelas dan objek kolaborasi yang mendukung use case. Oleh karena itu, setiap realisasi use case biasanya terdiri dari diagram kelas dan sejumlah diagram interaksi, yang paling umum adalah diagram sequence, yang menunjukkan kolaborasi atau interaksi antara objek fisik.

Realisasi use case memungkinkan analisis untuk secara jelas memisahkan masalah desain logis dan fisik. Karena desain logis (perilaku dan persyaratan bisnis) dapat diimplementasikan atau direalisasikan melalui sejumlah implementasi fisik yang berbeda, jika desain fisik berubah, kasus penggunaan logis tidak akan terpengaruh. Selain itu, realisasi use case adalah bentuk penelusuran persyaratan yang sangat baik mulai dari persyaratan bisnis logis hingga implementasi fisik dari solusi.

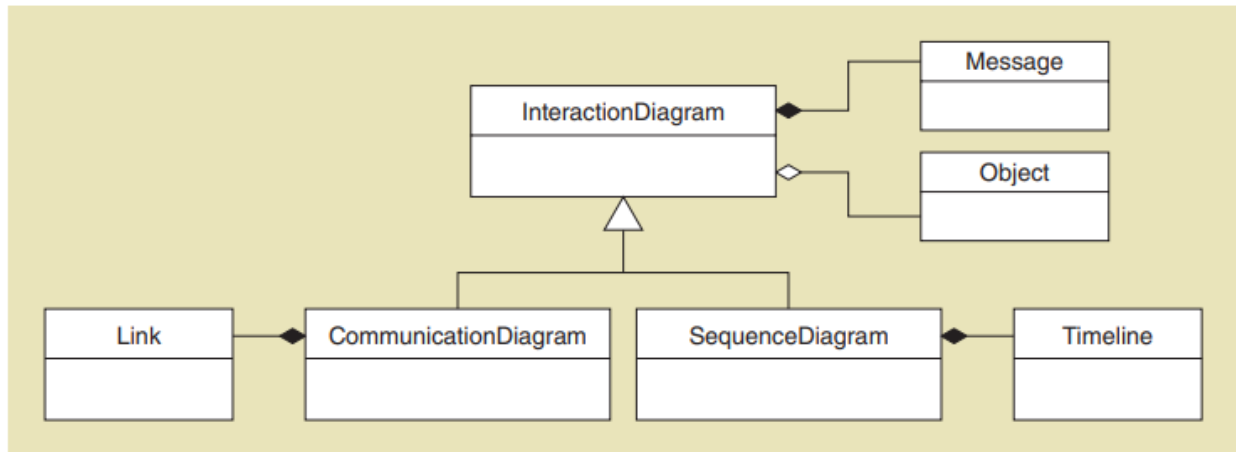
■ Object-Oriented Design with Interaction Diagram

Metode yang digunakan untuk memperluas proses detail desain disebut realisasi use case. Dalam realisasi use case, setiap use case diambil secara individual untuk menentukan semua class yang berkolaborasi di dalamnya. Sebagai bagian dari proses itu, setiap utilitas class atau dukungan lainnya diidentifikasi. Perawatan diambil selama proses ini untuk menentukan class sehingga integritas desain arsitektur multilayer dipertahankan.

Saat detail class didesain—use case by use case—desain diagram class juga diperbarui seperlunya. Selain kartu CRC, ada dua model kartu lainnya yang berguna untuk desain sistem—diagram komunikasi dan diagram sequence. Mengembangkan diagram interaksi adalah inti dari desain berorientasi objek. Realisasi use case—menentukan objek apa yang berkolaborasi dan pesan yang mereka kirim satu sama lain untuk menjalankan use case—adalah melalui pengembangan diagram interaksi. Dua jenis diagram interaksi dapat digunakan selama desain: yaitu diagram komunikasi dan diagram sequence.

Diagram komunikasi dan diagram sequence adalah jenis diagram interaksi pada UML. Gambar 9.2 adalah diagram class parsial yang menggambarkan fakta ini. Hal ini juga menunjukkan beberapa

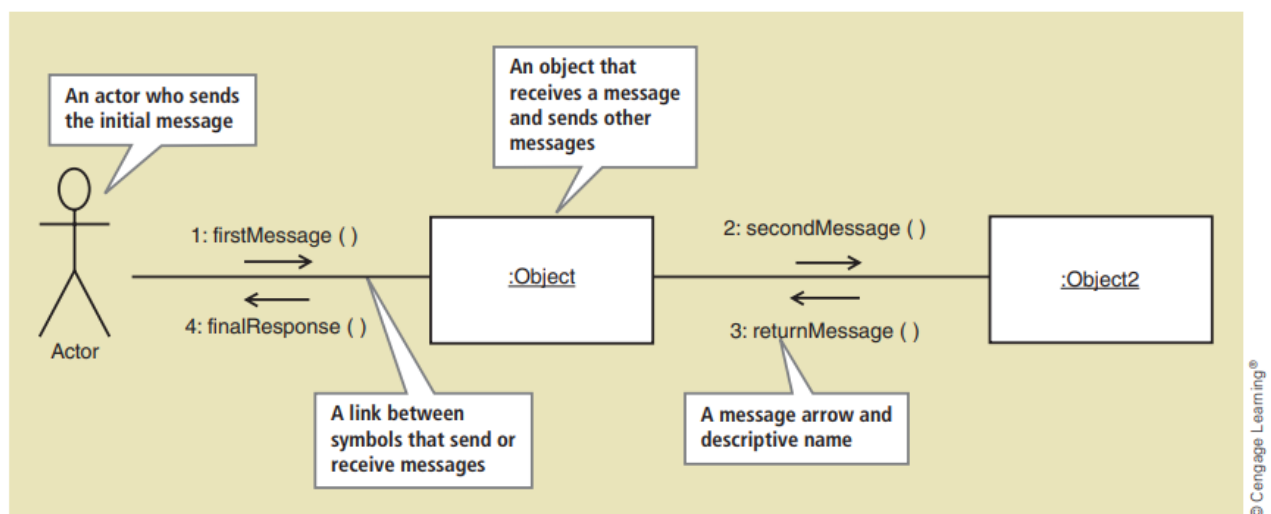
class komposisi untuk setiap class diagram. Setiap jenis diagram mampu menyediakan metode yang ketat untuk realisasi use case. Diagram interaksi, seperti namanya, fokus pada interaksi antara objek yang diperlukan untuk mengeksekusi use case tertentu. Interaksi antar objek disebut pesan. Metode ini menerjemahkan langsung ke metode class yang mendukung use case.



Gambar 9.2. Communication diagrams and sequence diagrams are interaction diagrams

■ Use case Realization with Communication Diagrams

Diagram komunikasi terdiri dari aktor, objek, tautan, dan pesan. Gambar 9.3 mengilustrasikan bagaimana berbagai komponen ini diilustrasikan dalam sebuah diagram.



Gambar 9.3. Symbols used in a communication diagram

Komponen-komponen tersebut memiliki ciri-ciri sebagai berikut:

- Actor
- Object
- Link
- Message

Pesan dalam diagram komunikasi memiliki arti yang sama. Namun, sintaks pesannya sedikit berbeda. Sintaksnya berisi lima elemen, yang semuanya opsional. Format komunikasi pesan diagram adalah sebagai berikut:

[true/false condition] sequence-number: return-value: 5 message-name (parameter-list)

- True/false condition
- Sequence number
- Return-value
- Message-name
- Parameter-list.

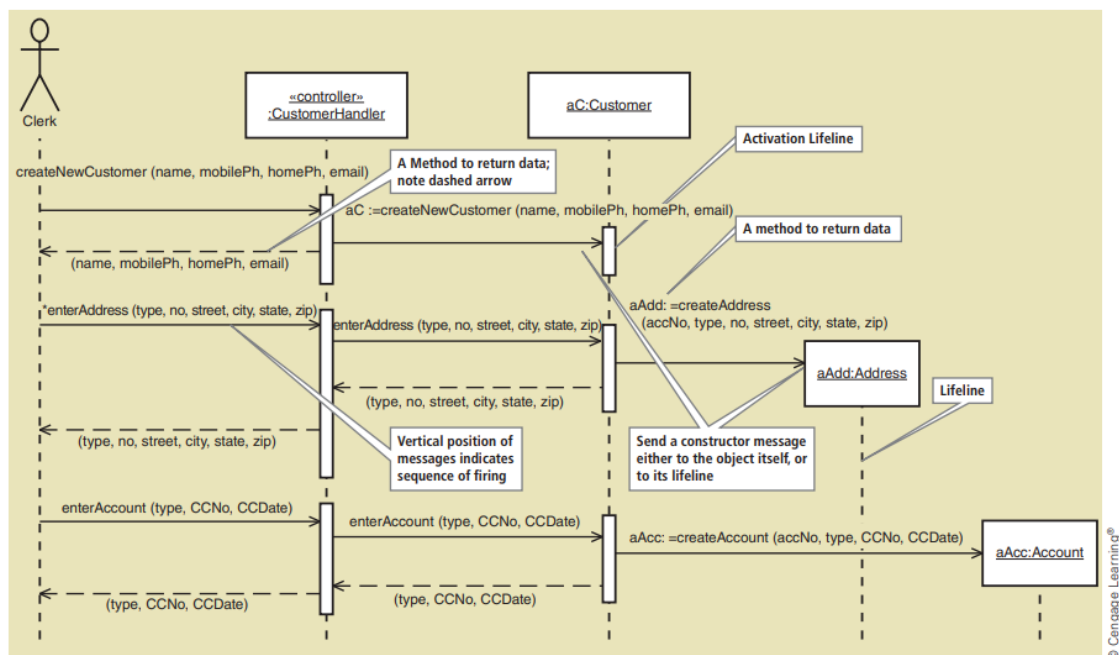
■ Use Case Realization with System Sequence Diagrams (SSD)

Informasi terpenting pada SSD adalah urutan pesan antara aktor dan sistem. Mungkin ada satu pesan yang memberi banyak masukan.

Pesan input mungkin memiliki parameter data atau tidak. Mungkin juga ada Loop frame, frame Alt, dan frame Opt serta input dan output yang berulang. Sebuah bingkai Loop menunjukkan satu set pesan dalam satu lingkaran. Bingkai Alt serupa pernyataan if-then-else atau pernyataan switch, yang memungkinkan penembakan set pesan yang berbeda. Bingkai Opt adalah pemanggilan opsional dari sekumpulan pesan bijak.

Gambar 9.4 adalah contoh diagram sequence. Dalam banyak hal, ini mirip dengan diagram komunikasi. Aktor merupakan peran eksternal, dalam hal ini Clerk. Beberapa objek diberi nama objek, seperti aC:Customer. Manfaat utama dari diagram urutan adalah kemampuan untuk meletakkan pesan dari atas ke bawah.

Di bawah setiap aktor dan objek adalah lifeline, yang digunakan sebagai indikator kehidupan benda tersebut. Pesan dilampirkan ke lifeline baik sebagai titik sumber atau titik tujuan. Terlampir ke lokasi dari setiap garis hidup adalah kotak vertikal yang mewakili garis hidup aktivasi.



Gambar 9.4. Sample sequence diagram from Create customer account use case

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

- Apa hubungan realisasi use case dengan diagram use case?
- Bagaimana bentuk hubungan realisasi use case dengan diagram-diagram UML seperti diagram aktifitas dan diagram sequence?
- Masing-masing use case akan di detailkan menjadi diagram aktifitas dan diagram sequence, bagaimanakah dengan realisasi use case, apakah mempengaruhi hal tersebut?

9.4 Multilayer Design

■ Developing a Multilayer Design

Sejauh ini dalam pengembangan diagram sequence, kita hanya berfokus pada class di lapisan domain masalah. Dalam banyak kasus, dokumentasi ini mungkin cukup untuk memprogram solusi—baik sendiri atau dengan programmer lain. Setelah Anda memiliki desain yang solid untuk class domain masalah, tambahkan lapisan tampilan akses data. Sesuai dengan prinsip pemodelan Agile, kami tidak ingin membuat diagram kecuali ada manfaat nyata. Kami juga biasanya tidak menyimpan desain diagramnya sebagai dokumentasi karena seiring waktu, sistem akan dimodifikasi dan diagram akan menjadi usang. Seperti yang disarankan oleh pemodelan Agile, berhati-hatilah dalam pengembangan model. Namun, ada kalanya penting untuk melihat kondisi system secara keseluruhan dan mengidentifikasi kebutuhan dan penggunaan class pada lapisan tampilan dan data yang mengakses class lapisan (layer). Seorang pengembang sistem perlu tahu bagaimana melakukan desain complete untuk contoh-contoh ketika diperlukan.

Setiap sistem akan membutuhkan class layer tampilan untuk mewakili layar input dan output untuk aplikasi. Class layer akses data tidak selalu diperlukan. Lapisan akses data diperlukan ketika logika bisnis cukup kompleks dan harus diisolasi dari pernyataan SQL yang mengakses database. CRC contoh kartu untuk kasus penggunaan akun Buat pelanggan ditunjukkan contoh desain tiga layer dengan class tampilan dan akses data (lihat Gambar 12-21). Karena kartu CRC adalah teknik informal, kami hanya menambahkan class tanpa banyak diskusi tentang bagaimana mereka terhubung ke class domain masalah. Bagian berikut mengembangkan contoh yang sama tetapi dengan diagram sequence. Teknik pemodelan yang lebih ketat ini menyoroti masalah yang terlibat dengan desain tiga layer. Pertama, kita tambahkan layer tampilan, lalu layer akses data.

■ Designing The View Layer

Salah satu keuntungan menambahkan diagram sequence adalah agar programmer dapat melihat bagaimana layer class tampilan terintegrasi dengan desain lainnya. Sekaligus memastikan apakah desain sudah benar dan lengkap. Ini adalah praktik yang baik untuk memverifikasi beberapa usecase untuk memastikan bahwa pengembang memahami bagaimana tampilan layer mewakili desain antarmuka pengguna dan bagaimana semua elemen terintegrasi bersama-sama untuk kelancaran program. Jadi, masukan untuk desain layer tampilan termasuk deskripsi usecase, SSD, diagram aktivitas, potongan pertama desain diagram class, dan akhirnya, tata letak antarmuka pengguna atau mock-up.

Desain antarmuka pengguna dan integrasi layer tampilan ke dalam urutan diagram dibuat lebih kompleks oleh fakta bahwa banyak sistem memerlukan antarmuka berbasis Web dan antarmuka berbasis jaringan internal. Untungnya, browser menjadi lebih canggih, sehingga banyak sistem baru dapat dirancang hanya untuk satu jenis antarmuka. Merancang sistem dengan banyak antarmuka pengguna adalah upaya yang cukup kompleks.

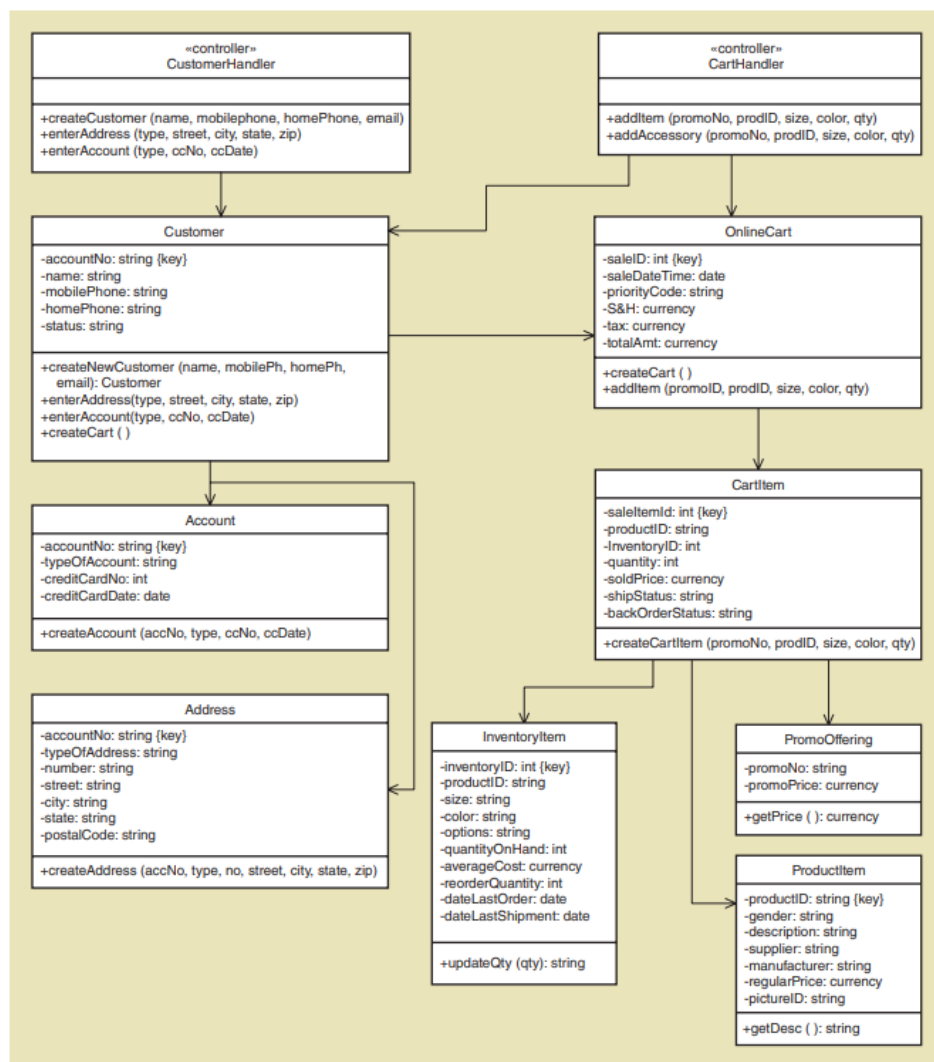
■ Updating and Packaging the Design Class

Desain diagram class terus tumbuh dan berkembang karena setiap usecase dirancang dan metodenya pun semakin bertambah. Contoh sebelumnya hanya membuat desain diagram class untuk class

domain masalah. Namun, desain diagram class juga dapat dikembangkan untuk setiap layer. Di layer tampilan dan lapisan akses data, beberapa class baru harus ditentukan. Layer domain juga memiliki class baru yang ditambahkan untuk pengontrol use case. Saat kami memperbarui desain diagram class, perhatikan bahwa ada tiga jenis metode yang ditemukan di sebagian besar class yaitu:

- metode konstruktor,
- metode pengambilan data dan kumpulan data,
- gunakan metode khusus kasus.

Metode konstruktor membuat metode baru untuk contoh objek. Gunakan metode tersebut untuk mengambil dan memperbarui nilai atribut. Untuk menghindari informasi yang berlebihan, sebagian besar pengembang tidak menyertakan get and set metode dalam DCD. Jenis metode ketiga—metode khusus usecase yang biasanya kami sertakan dalam desain diagram class. Gambar 5 berisi desain diagram class yang lengkap untuk class layer domain untuk dua usecase yang diilustrasikan adalah Buat akun pelanggan dan Isi keranjang belanja. Dua tambahan utama untuk class layer domain adalah dua usecase. Panah navigasi tambahan juga telah ditambahkan ke dokumen yang kelas terlihat dari class pengontrol. Panah navigasi lainnya, yang didefinisikan selama potongan pertama diagram class, telah terbukti cukup untuk dua usecase ini.



Gambar 9.5. Updated design class diagram for the domain layer

■ Design Patterns

Pola, juga disebut template, digunakan berulang kali dalam kehidupan sehari-hari. Seorang koki menggunakan resep, yang merupakan kata lain dari pola, untuk menggabungkan bahan-bahan menjadi hidangan beraroma. Seorang penjahit menggunakan pola untuk memotong kain untuk setelan yang pas. Insinyur mengambil komponen standar dan menggabungkannya ke dalam konfigurasi yang mapan, atau mengatur pola, untuk membangun gedung, sistem suara, dan ribuan produk lainnya.

Pola diciptakan untuk memecahkan masalah. Seiring waktu dan dengan banyak upaya, orang yang mengerjakan masalah tertentu mengembangkan solusi yang diberikan untuk masalah. Solusinya cukup umum sehingga dapat diterapkan berulang-ulang. Seiring berjalannya waktu, solusinya didokumentasikan dan diterbitkan, dan akhirnya diterima sebagai standar.

Template desain standar telah menjadi populer di kalangan pengembang perangkat lunak karena dapat mempercepat pekerjaan desain berorientasi objek. Nama resmi untuk template ini adalah pola desain. Pola desain menjadi teknik desain berorientasi objek yang diterima secara luas pada tahun 1996 dengan publikasi *Elements of Reusable Software* Berorientasi Objek oleh Erich Gamma, Richard Helm, Ralph Johnson, dan John Vlissides. Keempat penulis ini sekarang disebut sebagai Geng Empat (GoF). Saat Anda mempelajari lebih lanjut tentang pola desain, Anda akan sering melihat referensi ke pola desain tertentu sebagai pola GoF. Dalam buku mereka, penulis mengidentifikasi 23 pola desain dasar. Saat ini, sejumlah pola telah ditentukan—dari pola pemrograman tingkat rendah hingga pola arsitektur tingkat menengah hingga pola hadiah masuk tingkat tinggi. Dua platform perusahaan penting—Java dan .NET—memiliki set pola perusahaan, yang dijelaskan dalam berbagai buku dan publikasi. Anda mungkin juga akrab dengan konsep pola desain dan dengan dua pola spesifik: desain tiga lapis dan pengontrol usecase.

Pola ada di berbagai tingkat abstraksi. Pada tingkat konkret, sebuah pola dapat berupa definisi class yang ditulis dalam kode untuk digunakan oleh pengembang mana pun.

Belajar Mandiri



Pelajari lebih lanjut terkait Object-Oriented Design di materi PPT dan Video sesi 13 di LMS BINUS Online.

Jangan lupa juga untuk mempelajari materi pendukungnya.

Evaluasi Kelompok



Kerjakan Team Assignment 4 di LMS BINUS Online pada sesi 13 di LMS BINUS Online.

Post-Test

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

1. Beberapa pernyataan berikut yang tidak mewakili makna pengembangan dengan pendekatan Agile adalah...
 - a. Pendekatan pengembangan perangkat lunak yang fleksibel dan adaptif
 - b. Pendekatan pengembangan yang menekankan pada pembuatan model tanpa harus memiliki makna tertentu
 - c. Pendekatan pengembangan yang menekankan kolaborasi tim, komunikasi yang efektif, dan pengiriman perangkat lunak dalam iterasi pendek.
 - d. Pendekatan pengembangan yang memungkinkan perubahan kebutuhan pengguna selama proses pengembangan dan memungkinkan tim untuk merespons perubahan dengan cepat
2. Komponen berikut adalah komponen pada diagram komunikasi, kecuali...
 - a. Actor
 - b. Object
 - c. Line
 - d. Message
3. Seorang desainer yang tidak berpengalaman mungkin secara tidak sengaja memberi penekanan pada bagian halaman yang salah, yang justru akan membuat kebingungan di pihak pengguna, untuk itulah dibutuhkan emphasis.
 - a. Benar
 - b. Salah
4. Metode Pengembangan perangkat lunak berskala besar yang membutuhkan sistem yang kompleks dan melibatkan analisis mendalam mengenai tingkat risiko dan keberhasilan pengembangan disebut dengan metode DevOps.
 - a. Benar
 - b. Salah
5. Salah satu keuntungan bagi programmer untuk menambahkan diagram sequence dapat memastikan apakah desain sudah benar dan lengkap.
 - a. Benar
 - b. Salah

Kesimpulan

1. Use case realization adalah proses mengelaborasi desain rinci dengan diagram interaksi untuk usecase tertentu. Sementara itu, Communication diagram merupakan jenis diagram interaksi yang menekankan pada objek yang mengirim dan menerima pesan untuk tujuan usecase tertentu
2. Sequence diagram adalah jenis diagram interaksi yang menekankan urutan pesan yang dikirim antar objek untuk tujuan usecase tertentu.
3. Desain pattern adalah teknik desain standar dan template yang banyak diakui sebagai praktik yang baik. Desain antarmuka pengguna dan integrasi layer tampilan ke dalam urutan diagram dibuat lebih kompleks oleh fakta bahwa banyak sistem memerlukan antarmuka berbasis Web dan antarmuka berbasis jaringan internal.

Daftar Pustaka

1. Satzinger, John W., Jackson, Robert B., Burd, Stephen D. (2016). System Analysis and Design in a Changing World. 7th Edition. Cengage Learning, Boston USA. ISBN: 978-1-305-11720-4. Part 1, Chapter 13
2. Dina Fitria Murad (2024). Analysis, design, and development information system, PT. Widia Inovasi Nusantara, Jakarta, ISBN: 978-602-1138-84-7.
3. Pengembangan Perangkat Lunak Sistem Informasi,
<https://p2dpt.uma.ac.id/2024/04/30/pengembangan-perangkat-lunak-sistem-informasi/>
4. 11 Prinsip Desain dan Cara Penggunaannya, <https://binus.ac.id/knowledge/2021/12/11-prinsip-desain-dan-cara-penggunaannya/>

Post-Test

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

1. Beberapa pernyataan berikut yang tidak mewakili makna pengembangan dengan pendekatan Agile adalah...
 - a. Pendekatan pengembangan perangkat lunak yang fleksibel dan adaptif
 - b. Pendekatan pengembangan yang menekankan pada pembuatan model tanpa harus memiliki makna tertentu
 - c. Pendekatan pengembangan yang menekankan kolaborasi tim, komunikasi yang efektif, dan pengiriman perangkat lunak dalam iterasi pendek.
 - d. Pendekatan pengembangan yang memungkinkan perubahan kebutuhan pengguna selama proses pengembangan dan memungkinkan tim untuk merespons perubahan dengan cepat
2. Komponen berikut adalah komponen pada diagram komunikasi, kecuali...
 - a. Actor
 - b. Object
 - c. Line
 - d. Message
3. Seorang desainer yang tidak berpengalaman mungkin secara tidak sengaja memberi penekanan pada bagian halaman yang salah, yang justru akan membuat kebingungan di pihak pengguna, untuk itulah dibutuhkan emphasis.
 - a. Benar
 - b. Salah
4. Metode Pengembangan perangkat lunak berskala besar yang membutuhkan sistem yang kompleks dan melibatkan analisis mendalam mengenai tingkat risiko dan keberhasilan pengembangan disebut dengan metode DevOps.
 - a. Benar
 - b. Salah
5. Salah satu keuntungan bagi programmer untuk menambahkan diagram sequence dapat memastikan apakah desain sudah benar dan lengkap.
 - a. Benar
 - b. Salah